ARCHITECTURE-AWARE SPRING BOOT CODE GENERATION PLUGIN USING MACHINE LEARNING

Project Id: 25-26J-451

Project Proposal Report

Mahindarathna D.G.P.T.D

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

ARCHITECTURE-AWARE SPRING BOOT CODE GENERATION PLUGIN USING MACHINE LEARNING

Project Id: 25-26J-451

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

DECLARATION

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Mahindarathna D.G.P.T.D	IT22076052	

Signature of the Supervisor:
Date: 28/08/2025

Signature of the Co-Supervisor: S.S.Kumarasinghe 27/08/25
Date:

RESEARCH ABSTRACT

An estimation of more than 55% developers around the world tends to use spring boot as their preferred framework for backend development leading to a considerable increase in industry level usage of spring boot as their primary framework for server-side development. This high demand for spring boot framework in the industry intensifies the need for tools which preserve architectural consistency and project-specific conventions while accelerating development. Yet generic scaffolding engines and even advanced LLM code generators often ignore software architecture and the project-specific patterns and conventions producing codes which compiles but violates proposed code structure for relevant projects. Absence of spring boot specific code generating or scaffolding tools, lack of architecture aware code generation leading to a limited reliability on LLM code generators and lack of project-specific pattern and convention analysing tool to generate project-specific codes are considered as main issues in the current context.

The goal of this project is to develop an architecture aware project-specific code generation IntelliJ IDEA plugin using machine learning and static analysis to address the identified issues. The proposed system will take user input in a proper structure as YAML file and parse the inputs, parse the project-specific patterns and annotations, implement a statistical analysing tool which will extract the architecture related details from the codebase, predict the software architecture from the extracted data using machine learning algorithm, and generate accurate code based on the software architecture and parsed user inputs using a LLM.

Anticipated results include achieving at least ≥75% software architecture prediction accuracy on trained machine learning models using curated Spring Boot projects, higher compilation success of generated code addressing the gaps in the current context. By fusion of static analysis, machine learning, and LLM for code generation within an IDE-native workflow can conclude improvement in consistency and productivity for spring boot developers. In the end, the goal of this machine learning based architecture aware code generation plugin is to provide Spring Boot developers with an intelligent, architecture-aware assistant that generates consistent, project-specific code, improving productivity and maintainability.

Table of Contents

DECLARATION i

RESEARCH ABSTRACT ii

TABLE OF CONTENTS iii

1. INTRODUCTION 1

1. 1 Background & Literature Survey 1

1. 2 Research Gap 7

1. 3 Research Problem 9

2. OBJECTIVES 11

2. 1 Main Objectives 11

2. 2 Specific Objectives 11

3. METHODOLOGY 14

3. 1 System Architecture 14

3. 2 Software Solution 17

3. 3 Commercialization 20

3. 4 Future Scope 21

4. PROJECT REQUIREMENTS 22

4. 1 Functional Requirements 22

4. 2 Non-functional requirements 22

4. 3 System requirements 22

4. 4 User requirements 23

5. GANTT CHART 24

5. 1 Work Breakdown Structure (WBS) 24

6. BUDGET AND BUDGET JUSTIFICATION 25

REFERENCES 26

APPENDICES 29

List of Figures

Figure 1.1: Eclipse Foundation's Jakarta EE Developer Survey 1
Figure 1.2: ServiceNow 2024 Generative AI in Coding Survey 2
Figure 3.1 - Architecture aware code generation plugin System Architecture 14
Figure 3.2 - Agile Methodology [22]..... 18
Figure 3.3 - Sequence Diagram for Architecture-aware Spring Boot Code generation plugin 19
Figure 5.1 - GANTT CHART..... 24
Figure 5.2 - Work Breakdown Structure(WBS) 24

List of Tables

Table 1.1 - Comparison of Existing Spring Boot Code Generation / Scaffolding Tools..... 6
Table 1.2 - Comparison of Former Researches..... 7
Table 3.1 - Technologies, techniques, architectures, and algorithms used. 17
Table 6.1 - Budget Justification table 25

List of Abbreviations

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
EE	Enterprise Edition
GPT	Generative Pre-trained Transformer
IDE	Integrated Development Environment
JVM	Java Virtual Machine
LLM	Large Language Model
ML	Machine Learning
MVC	Model-View-Controller
REST	Representational State Transfer
SDK	Software Development Kit
SDLC	Software Development Life Cycle
UI	User Interface
UML	Unified Modeling Language

1. INTRODUCTION

1. 1 Background & Literature Survey

Java considered as the backbone of modern enterprise and cloud-native development, and with the high demand for java frameworks in the industry Spring Boot framework has emerged as the dominant framework in the java ecosystem [1]. As shown in the figure 1.1, the Eclipse Foundation's Jakarta EE Developer Survey reports [2] that Spring and Spring Boot frameworks are used by 66% of java developers building cloud native applications. This survey also retaliate that Spring Boot framework tends to replace Jakarta EE leveraging its specifications for portability and robustness.

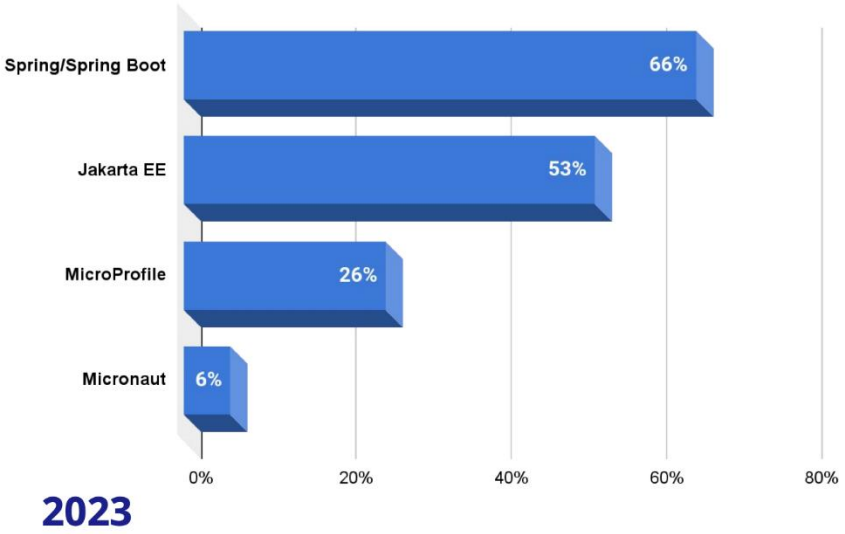

Figure 1.1: Eclipse Foundation's Jakarta EE Developer Survey

With this considerable rapid growth, organizations seek faster development cycles and delivering a production ready code within a short period of time ensuring consistence and quality development. Modern approach to achieve this goal is using LLM based code generation tools to simplify the development phase and testing phase and save a considerable amount of time [3]. Therefore, industry has an increased amount of need for faster scaffolding engines, architecture-consistent code generation tools and tooling that fits naturally into developers' IDE workflows. As shown as the figure 1.2 emphasize 72% of IT workers already using generative AI to enhance their performance in the development workflow [4]. Manual bootstrapping of spring boot

services consumes significant time and is error-prone specially when the development teams adhere to a chosen software architecture style for their projects across a large codebase. Since the current generative AI tools use Large Language Models capable of generating codes which will compile, lack of software architecture aware generation often leads to errors in the generated code comparing to the required output [5].

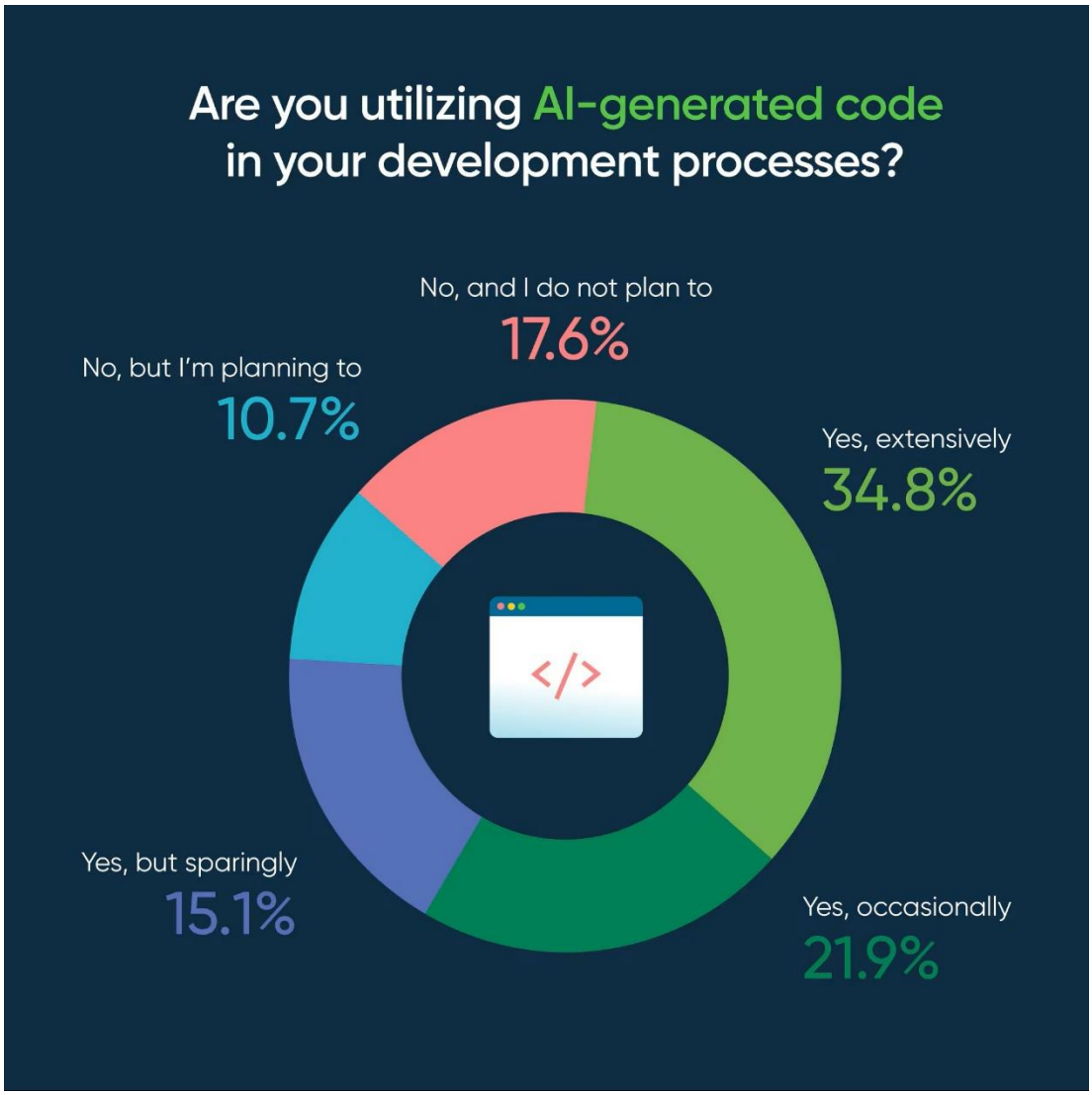

Figure 1.2: ServiceNow 2024 Generative AI in Coding Survey

With the high demand for LLM base code generation tools and increasing of the market for spring boot-based development, utilizing proper code generation by overcoming the main issues like architecture inconsistencies in code generations, not having proper code generation tools specifically for spring boot should be addressed. LLM base code generation tools available in the current market capable of extracting

design patterns from the existing projects / codebases [6]. But these LLM base code generation tools lack of capability to generate architecture-aware codes by extracting the software architecture from the relevant code repositories. Also, due to the lack of architecture-aware datasets to train these LLM models [5], generated codes are not accurate with the relevant software architectures used in the given projects. Moreover, there are significantly low number of code generation tools which specifically developed to generate spring boot code.

Literature Survey

Even though not many researchers addressed these constraints directly, there are several research addressed these constraints independently with considerable solutions. In *M. Mythily, A. Samson Arun Raj, and Iwin Thanakumar Joseph, "An Analysis of the Significance of Spring Boot in The Market"* [1], the authors emphasize Spring Boot as a preferred framework for backend development due to its scalability and multi-threaded capabilities. The study highlights Spring Boot's growing market adoption and underlines the need for tools and solutions that help developers achieve high-quality Spring Boot applications efficiently, aligning with research focused on accelerating development while preserving best practices.

In *"A Code Generation Framework for Java Component-Based Designs," Tsay, Hylands, and Lee (2000)"* [7], addresses the foundational tradeoff between programmer convenience and execution efficiency in component-based design systems. The authors proposed a solution using Java-based actor libraries with application graph analysis to understand components connection and data flows patterns. Through implementing a visitor design pattern for AST manipulation, they systematically transformed generic Java component into optimized target-specific code via three-stage pipeline: parsing, type resolution, and domain-specific optimization. The framework successfully addressed the core problem by achieving significant performance improvements while maintaining developer productivity, demonstrated through successful deployments on resource-constrained platforms like PalmOS devices. The research established key architectural principles for separating

domain-independent optimizations from domain-specific transformations, enabling extensible code generation that can adapt to different computational contexts while preserving the high-level abstraction benefits of component-based design.

In *"Generating Java Code from UML Class and Sequence Diagrams," Parada, Siegert, and Brisolara (2011)"* [8], addresses the fundamental challenge of automating Java application development through model-driven approaches. The authors tackle the complexity of manual Java coding processes that often lead to inconsistent code structures and prolonged development cycles, particularly relevant to enterprise Java applications where standardized patterns are crucial. The proposed solution employs a dual-diagram approach using UML class diagrams for Java class structure generation and sequence diagrams for method interaction modeling, processed through their custom GenCode tool that captures XMI-formatted UML models and automatically transforms them into executable Java code with proper object-oriented design principles. While the approach acknowledges limitations in generating complex business logic and fine-grained Java operations due to UML modeling constraints, the research established foundational principles for automated Java application development, providing high-level design abstraction benefits while maintaining compatibility with Java enterprise development practices.

Zhu and Zhang (2023) [9], explore the reliability of GPT-2 for automated code generation tasks, especially in challenging conditions. The study points out important weaknesses in pre-trained LLMs when they generate code that is syntactically correct but semantically incorrect. By conducting focused attacks, the authors show that GPT-2 can create compilable code that does not follow the intended program logic. This highlights the limitations in its understanding of architecture and context. The findings emphasize the need to include project-specific and architecture-aware rules in LLM-based code generation. This directly supports the development of IntelliJ plugin solutions that enforce Spring Boot architectural guidelines during automated code creation.

Nada et al. (2025) [10], explore the use of LLMs for synthetic data generation in both textual and code domains. The research highlights how generated code datasets can improve model training for downstream software engineering tasks, including code completion and automated scaffolding. Limitations include insufficient capture of project-specific patterns and architectural conventions, leading to potential inconsistencies in generated code. The findings support the integration of curated project datasets in training ML models for architecture-aware code generation, aligning with the research objective of analyzing Spring Boot projects to build a high-fidelity, pattern-preserving IntelliJ plugin.

Papadopoulos (2008) [11], provides a foundational exploration of automated code generation methodologies, focusing on translating high-level specifications into executable code. The study emphasizes the trade-offs between automation convenience and structural correctness, highlighting challenges in maintaining project-specific design patterns during code synthesis. These principles resonate with the proposed IntelliJ plugin, which integrates ML-based architecture prediction to ensure generated Spring Boot code adheres to established project conventions while minimizing developer effort.

Schmid et al. (2025) [12], conduct a systematic review on integrating LLMs with software architecture tasks, emphasizing the lack of tools that preserve architectural patterns in automated code generation. The review identifies critical challenges in maintaining consistency across layers, handling annotations, and enforcing project-specific conventions. These insights directly support the research objective of creating an IntelliJ plugin that combines static analysis, ML classification, and LLM-driven code generation to maintain architectural fidelity in Spring Boot projects.

The below Table 1 shows a comparison of existing Spring Boot code generation and scaffolding tools alongside the proposed IntelliJ plugin.

Table 1.1 - Comparison of Existing Spring Boot Code Generation / Scaffolding Tools

Tool/ Framework	**GitHub Copilot** [13]	**JHipster** [14]	**Spring Initializr** [15]	**Telosys Tools** [16]	**Proposed Intellij Plugin**
Approach	LLM-based code completion	Yeoman-based project scaffolding	Project starter generator (web UI)	Template-driven code generator	YAML + ML-based architecture-aware generation
Architecture Awareness	Low (pattern-agnostic)	Medium (microservices templates)	None	Low to Medium	High (pattern detection and architecture prediction)
IDE Integration	High (multi-IDE)	Medium (CLI/IDE)	Medium (IDE import)	Medium (Eclipse/IDEA plugin)	High (native IntelliJ)
Customizability	Medium	High	Low	High	High
Generated Code Quality	Variable; context-sensitive	Good, but rigid templates	Limited to initial boilerplate	Template-dependent	High-quality, architecture-consistent

The comparative analysis highlights that while tools like GitHub Copilot and JHipster improve productivity, they lack the ability to enforce project-specific architectural conventions. Spring Initializr and OpenAPI Generator provide helpful scaffolds but are limited in scope, often addressing only initial setup or REST APIs. Telosys offers custom template-driven code generation but requires significant manual intervention. In contrast, the proposed IntelliJ plugin integrates machine learning for architecture detection with YAML-driven code generation, ensuring that output code is not only consistent with developer workflows but also aligned with project-specific design patterns. This unique combination positions the plugin as a next-generation scaffolding tool for Spring Boot development.

1. 2 Research Gap

Automated code generation and scaffolding tools have seen extensive exploration in software engineering, focusing on productivity improvements, reduction of repetitive coding tasks, and enforcement of standard development practices. However, when analyzing existing research and tooling, significant limitations emerge in terms of architecture-awareness, integration into developer environments, and adaptability to project-specific conventions. The following prior works (Research A–E) demonstrate these constraints individually.

Table 1.2 - Comparison of Former Researches

Application Reference	**Architecture Awareness**	**Project-Specific Conventions**	**IDE/Tool Integration**	**ML/AI Usage**	**Maintainability Focus**
Research A	Medium	X	X	X	✓
Research B	Low	X	X	X	✓
Research C	Low	X	X	✓	X
Research D	Medium	X	X	✓	X
Research E	Medium	X	X	✓	X
Proposed Plugin	High	✓	✓	✓	✓

Research “A” proposed a code generation framework for Java component-based designs, focusing on actor-based modeling and domain-specific optimization [7]. While the study effectively improved performance on constrained devices, it primarily addressed execution efficiency rather than architectural fidelity in enterprise-level Spring Boot projects. The approach lacked adaptability to evolving frameworks like Spring Boot, leaving a gap in tools that balance both productivity and maintainable architecture.

Research “B” presented a system for generating Java code from UML class and sequence diagrams, enabling automated transformation of design diagrams into executable Java applications [8]. Although this method ensured structural consistency with UML models, it failed to capture complex business logic or Spring-specific

architectural conventions. Thus, while model-driven engineering accelerated initial development, the lack of framework-specific adaptability remains a gap.

Research “C” examined the robustness of large language models (LLMs) like GPT-2 for code generation [9]. Their findings revealed that while LLMs could generate syntactically correct code, outputs often contained semantic inaccuracies and failed to adhere to intended architectural patterns. This highlights a significant limitation of LLM-only approaches — the inability to enforce Spring Boot project structures and maintain architecture-aware generation.

Research “D” investigated synthetic data generation using LLMs for code and text, demonstrating the value of augmenting datasets to improve downstream tasks like automated scaffolding [10]. However, the generated datasets struggled to capture project-specific conventions and architecture fidelity, resulting in inconsistent outputs. This indicates a gap in integrating curated, architecture-labeled datasets into the generation process.

Research “E” conducted a systematic literature review on the use of LLMs for software architecture, identifying critical challenges such as maintaining consistency across layers and handling annotation-based frameworks like Spring [12]. While their review highlighted the lack of architecture-aware code generation tools, it did not propose a concrete implementation. This underscores the gap in building a practical, IDE-native solution that enforces architectural guidelines in real-world projects.

In summary, while past research has advanced productivity and automation in Java code generation, none have successfully combined machine learning-based architecture prediction, IDE-native integration, YAML-driven customization, and maintainability guarantees**.** The proposed IntelliJ plugin fills this void by enabling developers to automatically generate Spring Boot code aligned with project-specific architectural patterns, addressing shortcomings identified in Research A–E.

1. 3 Research Problem

We face intricate challenges in the pursuit of automating Spring Boot development through intelligent code generation. These challenges highlight the need for architecture-aware, customizable, and reliable solutions that integrate seamlessly into the developer's workflow. The following problems summarize the gaps motivating this research:

Lack of Architecture-Aware Code Generation

Current approaches to code generation, such as template-based or component-driven frameworks, focus primarily on developer convenience or execution efficiency [7] [8]. While effective in accelerating initial scaffolding, these tools fail to ensure adherence to architectural principles in enterprise-level projects. The absence of architecture-aware mechanisms results in generated code that often conflicts with layered, hexagonal, or clean architecture styles common in modern Spring Boot systems. This misalignment reduces maintainability, increases technical debt, and forces developers to manually refactor outputs to conform to architectural guidelines.

Lack of Code Generation Tool Focus on Spring Boot

Although Spring Boot has become a widely adopted backend framework due to its scalability and developer-friendly abstractions [1], most code generation tools remain framework-agnostic. Tools based on UML transformation [8] or component optimization [7] neglect the unique conventions of Spring Boot, such as dependency injection, annotation-driven configurations, and modular service definitions. As a result, developers lack specialized generation tools that exploit Spring Boot's strengths, limiting automation's value and leaving critical productivity gains unrealized.

Limitations of LLM-Only Code Generation

Recent advancements in large language models (LLMs) demonstrate their ability to generate syntactically correct code, but with notable shortcomings in semantic and architectural consistency [9]. Studies show that while models like GPT-2 can produce

compilable outputs, the generated code often deviates from intended design logic and fails to respect established architectural layers. This creates inefficiencies, as developers must spend considerable effort restructuring the generated scaffolds to align with Spring Boot conventions. Thus, relying solely on LLMs for code generation is inadequate, underscoring the need for hybrid approaches that incorporate architecture prediction and contextual guidance.

Absence of IDE-Native Integration

Despite recognition of usability as a critical factor for adoption, prior research has not produced a practical, IDE-integrated solution [12]. Most existing frameworks operate as external tools or standalone generators, requiring developers to step outside their established workflows. This lack of integration creates friction, reducing trust and hindering adoption. Particularly for Spring Boot developers who primarily use IntelliJ IDEA, the absence of an IDE-native plugin prevents seamless, architecture-aware generation from becoming a practical reality.

Inadequate Support for Project-Specific Conventions

Automated generation methods often rely on synthetic datasets or generalized design patterns [10] [11]. While useful for training, these approaches ignore project-specific rules such as naming conventions, annotation usage, and domain-driven entity structures. Consequently, generated outputs require significant manual refactoring to match the team's style and established conventions. The lack of YAML-driven customization or static analysis extraction exacerbates these inconsistencies, making existing tools poorly suited for real-world enterprise projects where uniformity and maintainability are critical.

Requirement for Accuracy and Maintainability

For code generation tools to be adopted in enterprise settings, they must guarantee both architectural correctness and compilation success. However, existing evaluations typically focus on generic correctness or productivity efficiency [7] [8] while overlooking architecture classification accuracy and post-generation reliability [9], [12]. Misclassifying a project's architecture leads to scaffolds that are not only

incorrect but also costly to repair. Without robust evaluation metrics, such as ML classification accuracy and code compilation success rates, the dependability of generated code remains uncertain. This undermines developer trust and prevents widespread adoption of automated generation techniques.

2. OBJECTIVES

2. 1 Main Objectives

The primary objective of this research is to develop an intelligent and architecture-aware code generation framework that streamlines the creation of maintainable and consistent Spring Boot applications. By combining machine learning-based architecture prediction with large language model–driven code generation, the study aims to produce outputs that not only compile successfully but also align seamlessly with established architectural conventions and project-specific requirements. Furthermore, the integration of the proposed solution into an IDE-native environment, specifically IntelliJ, seeks to enhance developer adoption, usability, and trust. Ultimately, this research endeavors to reduce developer overhead, improve productivity, and ensure sustainable software quality through automated, customizable, and context-aware code generation.

2. 2 Specific Objectives

There are six specific objectives that must be achieved to fulfill the primary objective described above.

1. **Develop ML-Based Architecture Prediction for Spring Boot Projects** This sub-objective focuses on creating a machine learning system capable of accurately classifying Spring Boot projects according to architectural patterns (e.g., Layered, Clean, Simple MVC). It involves extracting relevant structural features such as package depth, class relationships, annotation usage, and naming conventions, then training and validating a classifier to ensure high prediction

accuracy. Achieving this objective enables context-aware code generation that respects the project's architectural integrity.

2. **Build a Static Analysis Tool to Extract Architecture-Related Data**
The goal of this sub-objective is to develop a static analysis tool that scans Spring Boot codebases to extract structural and architectural information, such as class dependencies, package organization, annotations, and entity relationships. This tool will generate a structured dataset to feed the ML model and guide LLM-based code generation, ensuring that predictions and outputs reflect the actual project structure.

3. **Design a YAML-Driven User Input and Validation System**
The aim of this sub-objective is to provide a structured and user-friendly way for developers to specify entities, relationships, and configuration parameters in YAML files. The system will validate inputs to ensure consistency, prevent errors, and maintain alignment with Spring Boot conventions. This supports reliable automated code generation and reduces the need for manual corrections.

4. **Integrate LLM-Based Code Generation with Architecture Context**
This sub-objective targets the development of a code generation engine that combines the predicted architecture, project patterns, and validated YAML inputs to guide a large language model in producing Spring Boot code. The focus is on ensuring that generated code is semantically correct, adheres to architectural patterns, and respects project-specific conventions, minimizing developer intervention.

5. **Implement IDE-Native Plugin for IntelliJ IDEA**
The goal here is to seamlessly integrate the architecture-aware code generation system into IntelliJ IDEA. This includes building intuitive UI components, menu actions, and dialog windows that allow developers to interact with the plugin effortlessly. IDE-native integration aims to improve usability, increase adoption, and streamline development workflows.

6. **Evaluate Accuracy, Compilation Success, and Maintainability**
This sub-objective involves defining and applying metrics to assess the quality of generated code. Key measures include architecture prediction accuracy, code compilation success rates, adherence to conventions, and maintainability. By systematically evaluating these aspects, the research ensures the generated scaffolds are reliable, functional, and practical for real-world projects.

3. METHODOLOGY

3. 1 System Architecture

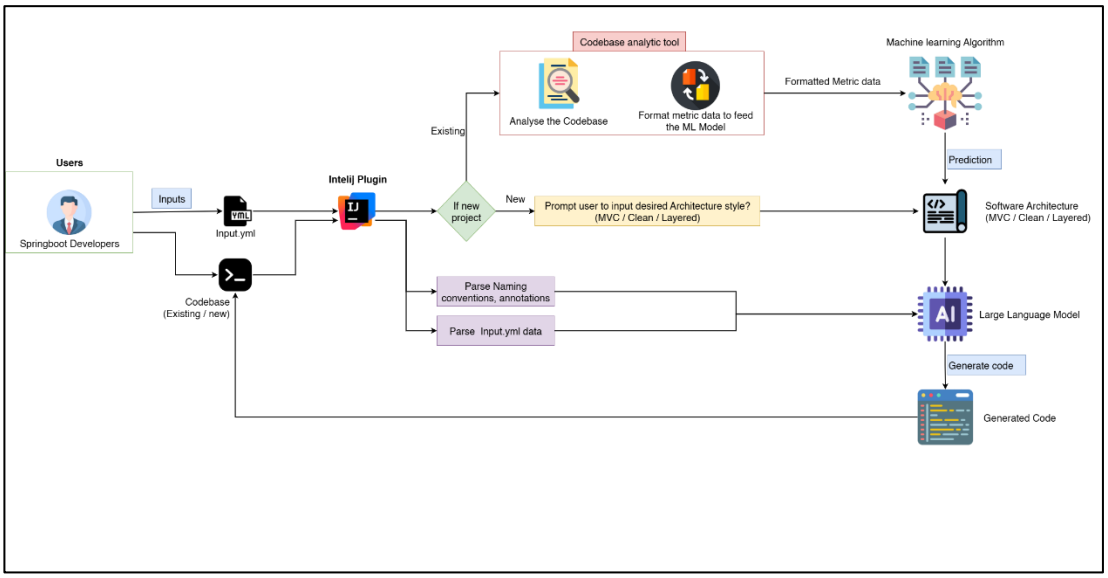

Figure 3.1 - Architecture aware code generation plugin System Architecture

Figure 3.1 illustrates the high-level software architecture of the proposed IntelliJ IDEA plugin for Spring Boot code generation. The architecture integrates static analysis, machine learning–based architectural prediction, and large language model (LLM)–driven code generation into a cohesive workflow that enables developers to generate architecture-aware, convention-compliant code with minimal manual intervention.

The process begins with the Spring Boot developer, who provides the initial inputs to the system. These inputs are supplied in the form of a YAML configuration file (input.yml), which captures the desired entities, attributes, and relationships for the project. The system is designed to accommodate both scenarios: working with an existing project or starting a new one from scratch.

When an existing project is provided, the plugin first interacts with a codebase analytic tool. This component is responsible for scanning the existing codebase to extract relevant architecture-related features. These features include class structures, annotations, dependencies, and naming conventions, which are then transformed into

structured metric data. The formatted data is passed into a machine learning model for classification**,** which predicts the architectural style of the existing project—whether it follows the Layered, Clean, or MVC pattern. This prediction ensures that the generated code aligns with the established project architecture.

In the case of a new project, the workflow differs slightly. Since there is no existing codebase to analyze, the plugin prompts the user to explicitly specify the desired architecture style (Layered, Clean, or MVC). This guarantees that the generated code conforms to the intended design from the very beginning.

Regardless of whether the project is new or existing, the system then proceeds to parse multiple sources of input. The plugin uses a YAML parser to extract entities and relationships defined in the input file, while another parsing mechanism analyzes project-specific conventions such as annotations and naming styles. These parsed elements, together with the architectural prediction (or user-specified architecture), serve as the contextual foundation for the next stage.

At the core of the system lies the Large Language Model (LLM) integration, which acts as the intelligent engine for code generation. The LLM receives all relevant inputs—entities and relationships from the YAML file, conventions from the existing project, and the predicted or specified architecture. Using this context, it generates code that is not only syntactically correct but also semantically consistent with the chosen architectural style. The generated code encompasses the main Spring Boot components such as entities, repositories, services, and controllers, ensuring that each component adheres to best practices while reflecting the project's conventions.

The output generation stage ensures that the generated code is directly written into the project structure within IntelliJ IDEA. Developers can immediately view, compile, and run the code without requiring manual restructuring or formatting. This seamless integration with the IDE makes the process highly interactive, providing real-time feedback and maintaining developer control over the generated artifacts.

The overall architecture thus emphasizes a balance between automation and customization. By combining static analysis for architecture detection, machine learning for predictive insights, and LLMs for context-aware code generation, the system ensures maintainability, architectural fidelity, and developer productivity. Furthermore, IntelliJ plugin integration provides a smooth workflow where developers can focus on higher-level design decisions, while repetitive and error-prone boilerplate code is automatically generated according to architectural and project-specific guidelines.

To support accurate architectural predictions, the system incorporates a dedicated model-training pipeline. Public Spring Boot projects are collected from GitHub and analyzed through the static analysis tool to extract architecture-related features such as dependencies, package structures, annotation usage, and naming conventions. These features are curated into a labeled dataset, which is then used to train three classification models:

1. A *Random Forest classifier* for robust baseline performance [17],
2. *XGBoost*, a gradient-boosted tree model known for its accuracy on heterogeneous feature sets [18],
3. A *feed-forward neural network* optimized through hyperparameter tuning for complex feature interactions [19].

Model selection is performed using cross-validation accuracy and per-class F1-score, with the best-performing model embedded within the IntelliJ plugin for real-time inference. This machine learning integration aligns with established research on software architecture recovery using static analysis and classification approaches [20] [21], ensuring that architecture predictions are both reliable and generalizable.

Summary of technologies, techniques, architectures, and algorithms used for the classification of Architecture-aware Spring Boot code generation plugin is shown in the table (Table 3.1) below.

Table 3.1 - Technologies, techniques, architectures, and algorithms used.

Technologies	Java, Spring Boot, Python, TensorFlow, GitHub, GitHub Rest API, Hugging Face Transformers/ OpenAI API, IntelliJ IDEA SDK, Docker, MongoDB, Maven, scikit-learn, YAML, Azure Machine Learning
Techniques	Data preprocessing & cleaning, Feature extraction, Supervised learning classification, Hyperparameter optimization, Prompt engineering, Model evaluation metrics, Data Curation, LLM fine-tuning, IntelliJ plugin packaging & UI integration, Containerization
Algorithms	Machine Learning Algorithms (Random Forest Classifier, XGBoost), Deep Learning Algorithm (Neural Networks with optimized hyperparameters), LLM-based code generation, YAML parsing & mapping algorithms
Architectures	Machine Learning Pipeline Architecture, LLM-based Code Generation Pipeline Architecture, Plugin Architecture, Client-Server Architecture

3. 2 Software Solution

The proposed IntelliJ IDEA plugin for Spring Boot code generation is developed following the principles of the Software Development Life Cycle (SDLC). To ensure adaptability and continuous improvement, the Agile methodology is adopted, as it allows iterative development, faster response to changing requirements, and frequent feedback from developers. Among the agile frameworks, Scrum is chosen due to its lightweight structure and efficiency in managing complex software projects. Figure 3.2 illustrates the Agile SDLC model adopted throughout the development of the IntelliJ IDEA plugin (see Agile SDLC Figure). This methodology ensures flexibility and iterative refinement tailored to the evolving technical demands of machine-learning-based plugin systems. Several studies have successfully applied Agile approaches in similar contexts—for example, Ranawana and Karunananda (2021) proposed an agile SDLC model for developing machine learning applications, emphasizing iterative integration of model training, evaluation, and deployment into software workflows [22].

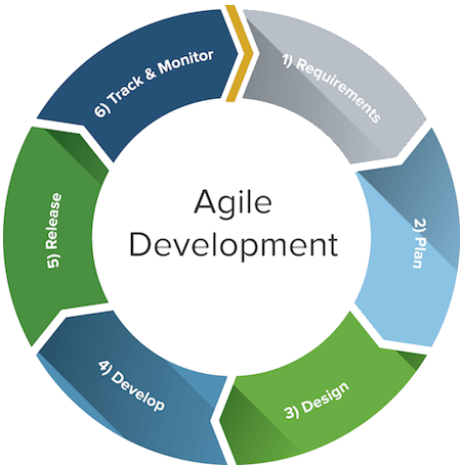

Figure 3.2 - Agile Methodology [22]

Requirement Gathering

Requirements for the solution were identified by analyzing the challenges faced by Spring Boot developers in generating architecture-consistent and convention-compliant code. These requirements were formalized into two categories: functional requirements, such as the ability to parse YAML files, predict architecture, and generate code through LLM integration; and non-functional requirements, including performance, maintainability, and seamless IntelliJ integration. Insights were also derived from existing literature on architecture recovery and automated code generation, further shaping the scope of the plugin.

Feasibility Study

Economic Feasibility – The proposed solution is cost-effective because it primarily builds on open-source tools, pre-trained large language models, and scalable cloud-based deployment of machine learning models. The expected benefit is reduced developer effort and minimized risk of architecture violations, which outweighs the development cost.

Scheduled Feasibility – Development is planned in iterative sprints, ensuring timely delivery of key features such as static analysis, model integration, and LLM-based code generation. The sprint-based approach reduces the risk of deadline failures by producing working increments at each iteration.

Technical Feasibility – The system is technically feasible as it integrates proven technologies: Python-based machine learning models (Random Forest, XGBoost, and Neural Networks), FastAPI for serving predictions, and IntelliJ IDEA plugin development using Java/Kotlin. Additionally, Azure deployment ensures scalability and secure access for model inference.

Design

The design phase covers both system and software-level architecture. The high-level architecture, presented in Figure 3.1, illustrates the interaction between components such as the YAML parser, static analysis tool, machine learning prediction engine, and LLM-driven code generator. Detailed design documents specify data flow, interface contracts between the plugin and external APIs, and compliance with Spring Boot architectural patterns (Layered, Clean, MVC).

Sequence Diagram

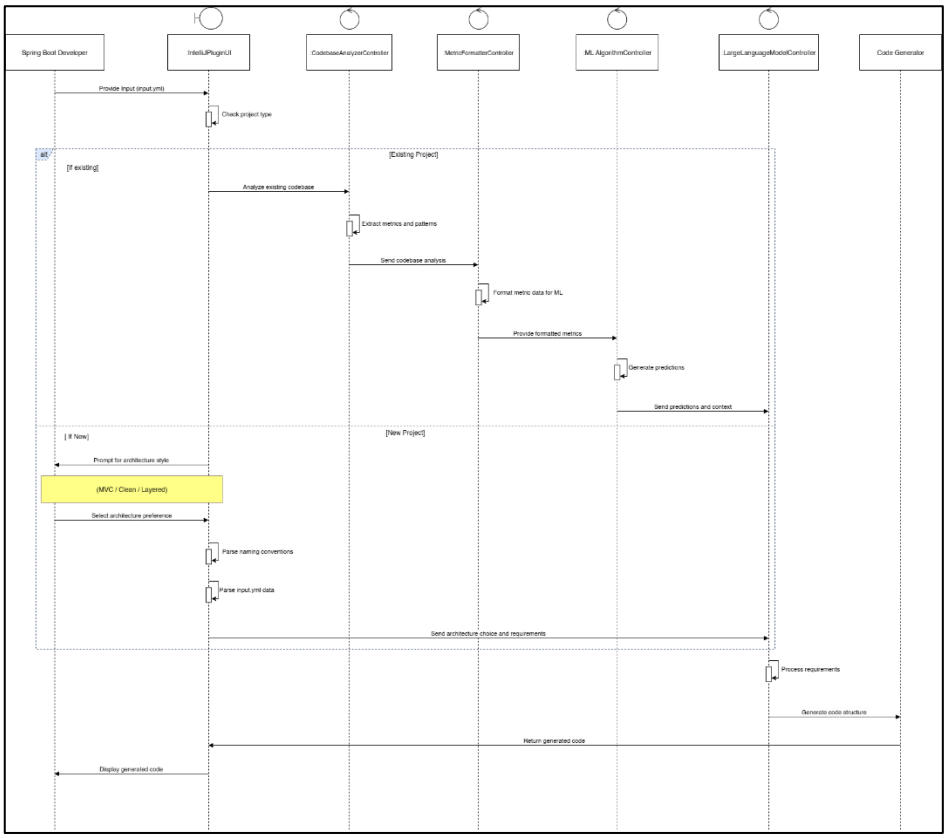

Figure 3.3 - Sequence Diagram for Architecture-aware Spring Boot Code generation plugin

Implementation (Development)

Implementation follows incremental development cycles, where each sprint adds functionality to the IntelliJ plugin. Initial versions focus on YAML parsing and static analysis, followed by the integration of the trained machine learning model deployed in Azure. Subsequent iterations incorporate LLM-based code generation, ensuring that the generated entities, repositories, services, and controllers are architecture-aware and convention-compliant. The final stage integrates all components into IntelliJ IDEA, providing real-time developer feedback.

Testing

A rigorous testing strategy ensures software quality and reliability. Unit testing is applied to plugin components such as the YAML parser and code generators, while integration testing validates communication between the IntelliJ plugin and the cloud-deployed ML model. System testing ensures the correctness of architecture predictions and code generation, while user acceptance testing involves developers validating usability and practical performance within real-world Spring Boot projects.

3. 3 Commercialization

- **Developer Productivity Market**

  The plugin can be positioned as a code generation and refactoring accelerator for enterprise developers using IntelliJ IDEA, which is widely used in Java, Kotlin, and JVM-based projects.

- **Integration with CI/CD Pipelines**

  Beyond the IDE, the plugin can expose APIs for Jenkins, GitHub Actions, or Azure DevOps, offering automatic architecture compliance checks during code commits. This turns the tool into not just an IntelliJ plugin but an enterprise DevOps assistant.

- **Enterprise Licensing**

  Offer enterprise-grade licensing with custom-trained models for organizations that want to encode their own software design rules (naming conventions, annotations, and internal architectural constraints).

  Hosted on Azure private endpoints for secure corporate deployment.

3. 4 Future Scope

Future work envisions advancing the development assistant from syntactic-level code generation to semantic-level architecture validation, ensuring that generated code not only compiles successfully but also conforms to established design patterns, microservice communication protocols, and performance optimization guidelines. This can be further enhanced by fine-tuning large language models on real-world open-source projects to improve domain adaptability and context-aware suggestions. While the initial implementation targets IntelliJ IDEA, expanding cross-IDE compatibility to platforms such as VS Code, Eclipse, and other JetBrains IDEs like PyCharm and WebStorm is a key objective. Moreover, not only limited to Spring Boot specific code generation, but this plugin can also be enhanced to generate compilable code based on user preferred programming languages by further fine tuning LLMs.

4. PROJECT REQUIREMENTS

4. 1 Functional Requirements

1. Develop a static tool to extract architecture related data from codebase.
2. Train a classification machine learning/ deep learning models from curated spring boot project related dataset for architecture prediction
3. Extract user requirements, naming conventions and project patterns from input.yml file and from the codebase
4. Fine tune a LLM to generate code based on user requirements, naming conventions, project patterns and predicted architecture.
5. Integrate a user feedback system to analyse the success rate of generated code.

4. 2 Non-functional requirements

1. Accuracy – Since the system is predicting the software architecture, which is critical for proper code generation, predicted outcome should be accurate.
2. Performance – System should perform efficiently by providing fast predictions and fast generation of accurate codes.
3. Reliability – System should not fail or get stuck at anytime throughout the process.

4. 3 System requirements

The purpose of software requirements is to specify the necessary resources that a system must implement to ensure the proper functioning of the proposed system. The software specification requirements for this component are outlined as follows.

- Java and IntelliJ IDEA SDK to create the Plugin structure.
- TensorFlow and scikit-learn for machine learning and deep learning models.
- GitHub Rest API for data collection.
- GitHub for version control.
- Python for machine learning/ deep learning model development
- Hugging Face Transformers / OpenAI LLM for code generation
- YAML for input (entity relation) parsing
- Trello / Jira for project management

4. 4 User requirements

This plugin is mainly developed for two types of users.

1. Native Spring Boot developers
   Spring Boot developers will use this plugin to automate their coding projects with automated code generation. The plugin will ensure compilable architecture-aware generated code for better automation.

2. Students
   Students who started to learn Spring Boot will use this plugin to generate complex codes based on entity relations by giving simple inputs in the input.yml and get a proper accurate generated code.

5. GANTT CHART

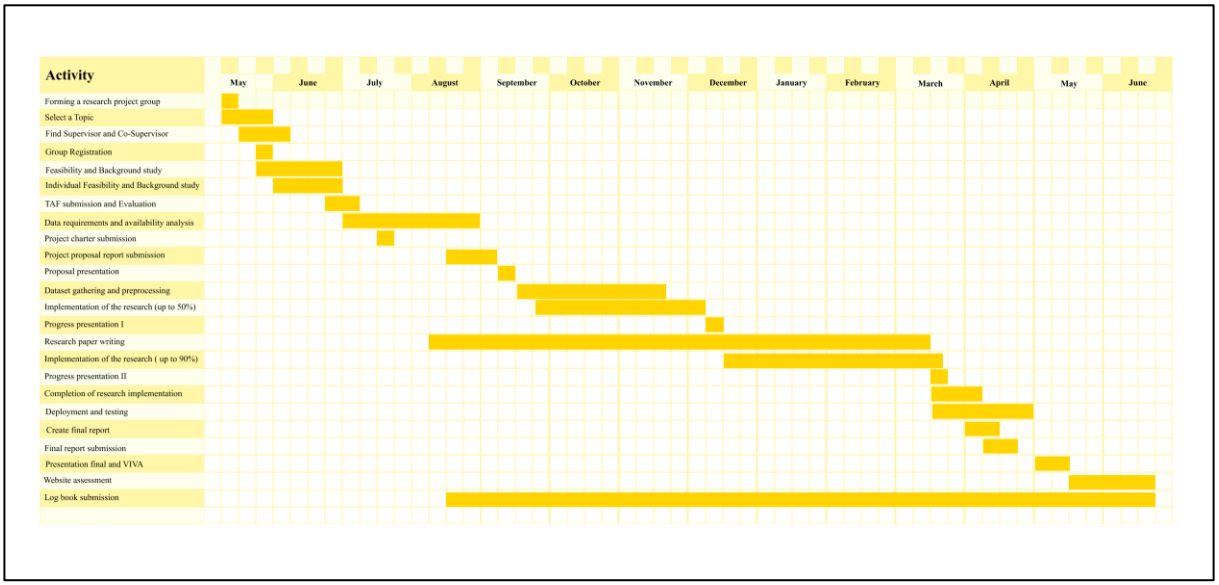

Figure 5.1 - GANTT CHART

5. 1 Work Breakdown Structure (WBS)

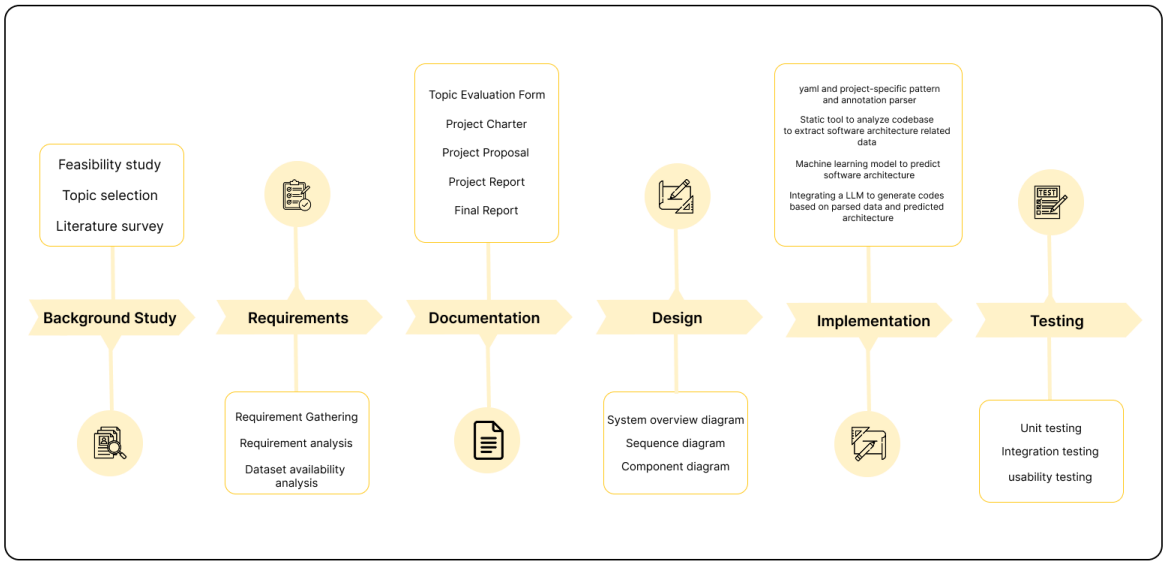

Figure 5.2 - Work Breakdown Structure(WBS)

6. BUDGET AND BUDGET JUSTIFICATION

The below Table 6.1 depicts the overall budget estimation for the proposed IntelliJ IDEA plugin system, including costs for dataset preparation, model training, deployment, and operational expenses.

Table 6.1 - Budget Justification table

Requirement	**Cost (LKR)**
Cost for data collection and preprocessing (GitHub projects, curation, annotation)	0.00 (Using publicly available spring boot repositories to curate the dataset manually)
Internet Charges	Rs. 5000
Total Cost	Rs. 5000

REFERENCES

[1] M.Mythily, A.Samson Arun Raj and Iwin Thanakumar Joseph, "An Analysis of the Significance of Spring Boot in The Market," in *2022 International Conference on Inventive Computation Technologies (ICICT)*, 2020.

[2] E. Foundation, "2023 Jakarta EE Developer Survey Report," Eclipse Foundation, Ottawa, Canada, 2023.

[3] Z. L. Juhász, "Large Language Models, Fine Tuning, Code Generation," in *IEEE 7th International Conference and Workshop in Óbuda on Electrical and Power Engineering (CANDO-EPE)*, Budapest, Hungary, 2024.

[4] ServiceNow, "72% of IT Workers Are Embracing Generative AI: The Path Toward Human-AI Collaboration," ServiceNow, Santa Clara, California, USA, 2024.

[5] E. Matteo , L. Xiaozhou, M. Sergio , A. Noman, C. Tomas , V. Karthik, L. Valentina and T. Davide, "Generative AI for Software Architecture. Applications, Trends, Challenges, and Future Directions," 2025.

[6] C. Rongyu, L. Binhua, L. Yongbin and L. Han, "Do Code LLMs Understand Design Patterns?," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 2025.

[7] Jeff Tsay, Christopher Hylands and Edward A. Lee, "A Code Generation Framework for Java Component-Based Designs," in *CASES 2000 (ACM Conference)*, San Jos, 2000.

[8] Abilio G. Parada, Eliane Siegert and Lisane B. de Brisolara, "Generating Java Code from UML Class and Sequence Diagrams," in *2011 Brazilian Symposium on Computing System Engineering*, Florianópolis, Brazil, 2011.

[9] R. Zhu and C. Zhang, "How Robust Is a Large Pre-trained Language Model for Code Generation? A Case on Attacking GPT2," in *30th Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*, Lisbon, Portugal, 2023.

[10] M. Nada, L. Dioșan and A. Tomescu, "Synthetic Data Generation Using Large Language Models: Advances in Text and Code," *IEEE Access*, vol. 13, p. 3589503, 2025.

[11] G. A. Papadopoulos, "Automatic Code Generation: A Practical Approach," in *30th Int. Conf. on Information Technology Interfaces (ITI)*, Cavtat, Croatia, 2008.

[12] L. Schmid, M. Armbruster, D. Fuch and H. Liu, "Software Architecture Meets LLMs: A Systematic Literature Review," arXiv, Online, 2025.

[13] GitHub, "GitHub Copilot Documentation," GitHub, Available at: https://docs.github.com/en/copilot, 2024.

[14] D. K. a. D. J. Sasidharan, "JHipster: Open Source Application Generator for Modern Web Apps," JHipster Project, Available at: https://www.jhipster.tech, 2023.

[15] Spring.io, "Spring Initializr Documentation," Spring.io, Available at: https://start.spring.io, 2024.

[16] T. Project, "Telosys Tools Documentation," Telosys.org, Available at: http://www.telosys.org, 2024.

[17] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, p. 5–32, 2001.

[18] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016.

[19] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, p. 281–305, 2012.

[20] C. Izurieta, J. M. Bieman and R. A. Malhotra, "Design pattern decay: A longitudinal study of design pattern maintenance," in *Proceedings of the IEEE International Conference on Software Maintenance (ICSM)*, Timisoara, Romania, 2010.

[21] A. Sarkar, S. Saha and R. K. Saha, "Machine learning for software architecture recovery: A systematic mapping study," in *IEEE/ACM International Conference on Software and System Processes (ICSSP)*, Seoul, South Korea, 2020.

[22] R. Ranawana and A. S. Karunananda, "An Agile Software Development Life Cycle Model for Machine Learning Application Development," in *2021 5th SLAAI International Conference on Artificial Intelligence (SLAAI-ICAI)*, University of Kelaniya, Sri Lanka, 2021.

APPENDICES